

Build a Tiny Captioning VLM

A frozen vision encoder, a learned adapter, and a small decoder -- trained live, on a laptop.

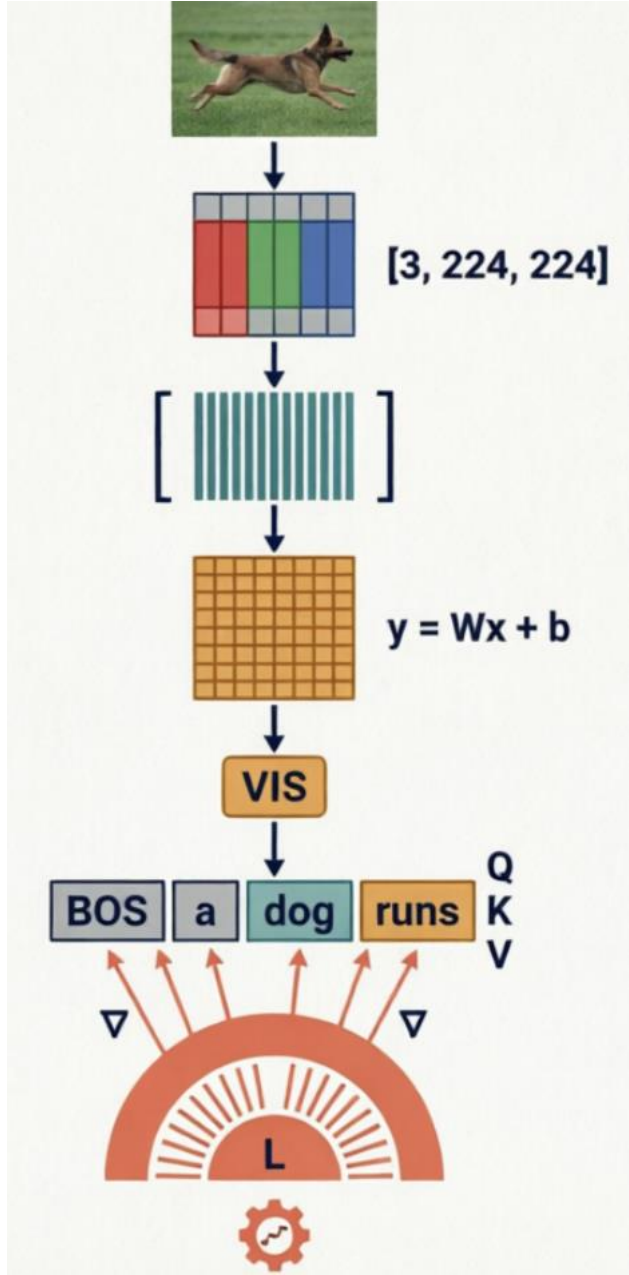
“From scratch” only for the two pieces that actually train here.

LEARNING OUTCOMES

One tensor's journey, start to finish

By the end of this talk you should be able to trace one image, as a tensor, through every stage below -- and connect each stage back to Part 1's mathematics.

Part 1 concept	Where it shows up today
$y = Wx + b$	The two linear layers inside the projector
Embeddings	CLIP's pooled vector; the decoder's token embeddings
Attention	Causal self-attention over [visual token, caption tokens]
Loss minimization	Cross-entropy on the next caption token
Backpropagation	<code>loss.backward()</code>
Adam	<code>torch.optim.AdamW</code>



STEP 0 • BEFORE ANYTHING ELSE

Start training now. We'll watch it while we talk.

```
$ ./hardware.sh
tinyVLM hardware + environment check
-----
OS           : Darwin 25.6.0 arm64
CPU          : Apple M4
CPU cores    : 10 physical / 10 logical
RAM total    : 32 GB
  -- RAM mostly matters for image decoding + CLIP feature caching, not
      model size: the trainable parameters here fit in a few hundred MB.
GPU / accel  :
      Chipset Model: Apple M4
      Metal Support: Metal 4
```

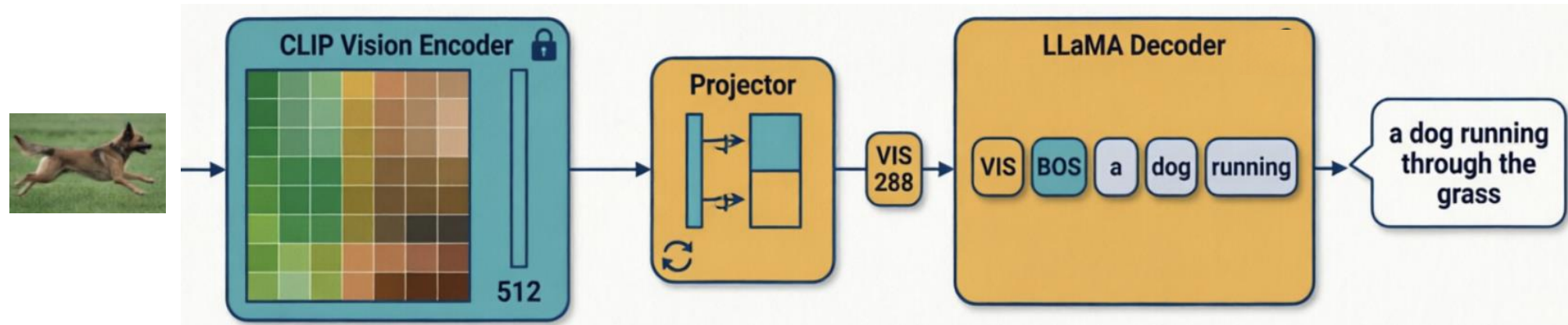
```
$python3 trainVLM.py --config config.demo.yaml
```

```
# THE exact command timed for this talk: 350 steps, batch size 16,
# a fixed 1,500-image training subset (repeated ~3.7x) -- see config.demo.yaml
```

Dataset, CLIP weights, and the holdout demo images are already cached on this machine.

THE WHOLE MODEL, WITH SHAPES

Image in, sentence out



pixels	$[B, 3, 224, 224]$	<- one photo, batch of B
CLIP embedding	$[B, 512]$	<- pooled, frozen, cached per image
visual token	$[B, 1, 288]$	<- projector output
text embeddings	$[B, T-1, 288]$	<- BOS + caption so far
decoder input	$[B, T, 288]$	<- concat(visual token, text embeddings)
logits	$[B, T-1, V]$	<- see 'teacher forcing' slide for the T-1

Pixels: $B \times 3 \times 224 \times 224$

CLIP: $B \times 512$

Visual prefix: $B \times 1 \times 288$

Decoder input: $B \times T \times 288$

Training logits: $B \times (T-1) \times V$

Gray = input, not a model. Teal = frozen. Amber = the two things that train. This is visual-PREFIX self-attention, not cross-attention -- there's one sequence, no separate image keys/values.

Split by image, not by caption

```
image      : 1000268201_693b08cb0e.jpg
caption_0  : a child in a pink dress is climbing up a set of stairs
caption_1  : a girl going into a wooden building
caption_2  : a little girl climbing into a wooden playhouse
caption_3  : a little girl climbing the stairs to her playhouse
caption_4  : a little girl in a pink dress going into a wooden cabin
```

Five captions, one photo. They're near-duplicates of each other -- which is exactly why the split has to happen at the **IMAGE** level, before any caption is picked.

TRAIN

~6,000 images official
(1,500-image subset for this demo)

VALIDATION

~1,000 images
picks best.pt

TEST / HOLDOUT

~1,000 images
never trained on

Configuration parameters : data

Field	Meaning
dataset	HuggingFace dataset id (<code>jxie/flickr8k</code>), passed to <code>load_dataset()</code>
cache_dir	Where the downloaded images/captions get cached locally
image_size	Documents CLIP's expected input size (224×224); the actual resizing happens inside <code>CLIPImageProcessor</code>, not read directly from here
max_caption_len	Caption token cap, including <code><bos>/<eos></code> — longer captions get truncated
seed	Used for the train-subset shuffle and the holdout-image selection, so both are reproducible run to run
train_subset_size	The lecture-specific trick: train on only this many images (<code>1,500</code>) instead of the full $\sim 6,000$, so the CLIP feature cache actually gets reused within <code>350</code> steps (see the file header comment)
export_holdout	Whether to write demo images + references + manifest to disk on each run
holdout_dir	Where those go (<code>inferVLM.py --holdout-index</code> reads from here)
holdout_n	How many holdout images to export

Turning text into padded integer tensors

- 1

Character-level tokenizer, by default

no additional tokenizer model download; vocab built from the training captions themselves (~75 symbols).
- 2

One caption sampled per image per epoch

a different one of the five each epoch -- free augmentation, no expanded dataset.
- 3

Pad to the longest caption in the batch

no attention mask needed: right-padding + causal attention can't leak, and the loss ignores pad positions directly.



Sample	Original caption	Character tokens	Padded tensor row	Loss evaluated on
A	a dog	<bos> a d o g <eos>	<bos> a d o g <eos> <pad> <pad> <pad>	a d o g <eos>
B	two cats	<bos> t w o c a t s <eos>	<bos> t w o c a t s <eos>	t w o c a t s <eos>

Configuration parameters : projector

Field	Meaning
freeze_decoder	false = decoder trains too (the default);
batch_size	Images per gradient step (16)
epochs / max_iters	max_iters is what actually stops the run (350);
lr	Peak learning rate for AdamW
weight_decay	AdamW's L2-like regularization strength
grad_clip	Max gradient norm before clipping, for stability
warmup_iters	Steps over which lr ramps up linearly before the cosine decay (see get_lr())
log_every	How often to print/log a training-loss row
eval_every	How often to run validation and possibly save best.pt
ckpt_every	How often to save a numbered checkpoint + last.pt
out_dir	Where checkpoints go
metrics_csv	Where the CSV visualize.py reads gets written; blank = <out_dir>/metrics.csv
device	auto picks cuda > mps > cpu
dtype	Documented but not currently switched on (training always runs in float32)
num_workers	DataLoader worker processes for image loading

Configuration parameters : decoder

Field	Meaning
dim	Embedding/hidden dimension {288}
n_layers	Transformer blocks stacked {6}
n_heads / n_kv_heads	Attention heads; equal here means ordinary multi-head attention (grouped-query attention only kicks in if n_kv_heads < n_heads)
max_seq_len	Longest sequence the rotary position embeddings are precomputed for
multiple_of	Rounds the SwiGLU MLP's hidden size up to a multiple of this, matching llama2.c's convention
dropout	0 here — no regularization needed at this scale/step count
init_checkpoint	Blank by default (random init); optionally a path to a llama2.c stories15M.bin for the opt-in comparison experiment
tokenizer	Blank by default; optionally a SentencePiece tokenizer.model path, required if you use init_checkpoint
fallback_tokenizer	What to use when tokenizer is blank — the character-level tokenizer

TinyVLM.forward(), line by line

```
img_feat      = clip_features or self.encode_image(pixel_values) # [B, 512]
visual_tok    = self.projector(img_feat)                        # [B, 1, 288]
tok_embeds    = self.decoder.tok_embeddings(caption_ids[:, :-1]) # [B, T-1, 288]
x             = cat([visual_tok, tok_embeds], dim=1)           # [B, T, 288]
logits        = self.decoder(x, is_embeds=True)                # [B, T, V]
return logits[:, 1:, :]    # <- aligned with caption_ids[:, 1:]; see next slide
```

Sequence fed to the decoder: [visual, BOS, tok₁, ..., tok_{T-2}]. **Decoder position *i* predicts sequence position *i*+1 --**

Three lines most of the mathematics lecture was about

```
loss.backward()           # backprop: d(loss)/d(every trainable weight)
torch.nn.utils.clip_grad_norm_(trainable, 1.0)
optim.step()              # AdamW: adaptive per-parameter updates
```

What's actually trainable?

Decoder: ~6.0M params (char tokenizer)

Projector: 410,400 params

Both computed by `build_model()` at startup

```
[model] parameters --
  frozen: 88,000,000
  decoder: 5,995,872
  projector: 410,400
  trainable total: 6,406,272
```

Training loss going down is necessary. Not sufficient.

```
step 150/350 | loss 3.21 | lr 2.9e-04 | 41.2 examples/s | 58.1s elapsed | ETA 78s
step 160/350 | loss 3.09 | lr 2.8e-04 | 42.0 examples/s | 61.5s elapsed | ETA 70s
      [eval] val_loss 3.34 (val feature cache: 320 images cached)
      [ckpt] new best -- saved checkpoints/best.pt
```

Plot both curves, not just training loss: smoothed train loss, validation loss at each eval point, and a marker for whichever checkpoint is currently best.pt. This demo trains on a fixed 1,500-image subset for ~3.7 passes -- watch for validation loss stalling or rising while training loss keeps falling, the expected sign of overfitting on a small, repeated subset.

MID-TALK CHECK-IN

Let's flip back to that terminal

```
step   240/350 | loss 2.51 | lr 1.9e-04 | 43.8 examples/s | 92.3s elapsed | ETA 25s  
      [eval] val_loss 2.88  
      [ckpt] saved checkpoints/step200.pt
```

Loss trending down, a best.pt on disk, an ETA in the tens of seconds -- that's everything the rest of the talk needs to be true. Back to the slides.

Training sees the whole caption. Inference doesn't.

Training	Inference
Whole caption available at once	Only the generated prefix so far
Teacher forcing	Autoregressive feedback, one token back in
Cross-entropy loss	Greedy (default) or temperature/top-k sampling
Many positions scored in parallel	One new token computed at a time

This asymmetry is exactly why the teacher-forcing alignment bug mattered: training could look fine while inference -- which never gets the luxury of the true next token -- produced nonsense.

One test-split photo, greedy decoding, reproducible

```
$ python3 inferVLM.py --config config.demo.yaml --checkpoint checkpoints/best.pt --holdout-index 0
```



```
image      : demo/holdout_0000.jpg
decoding    : greedy (reproducible)
caption     : a dog running through the grass
references  :
  - a brown dog runs through the green grass
  - a dog is running in a field
```

An unseen image tests generalization -- one plausible caption next to the five human references is qualitative evidence, not proof. Greedy decoding is the default here specifically so this exact command reproduces this exact caption every time.

RECAP

The tensor's whole journey, in five steps

1

CLIP represents the image

a 512-dim vector, computed once, cached, never updated.

2

The projector produces a visual prefix

the only new, from-scratch idea in this build.

3

The decoder predicts caption tokens

causal self-attention over [visual, caption-so-far], random init.

4

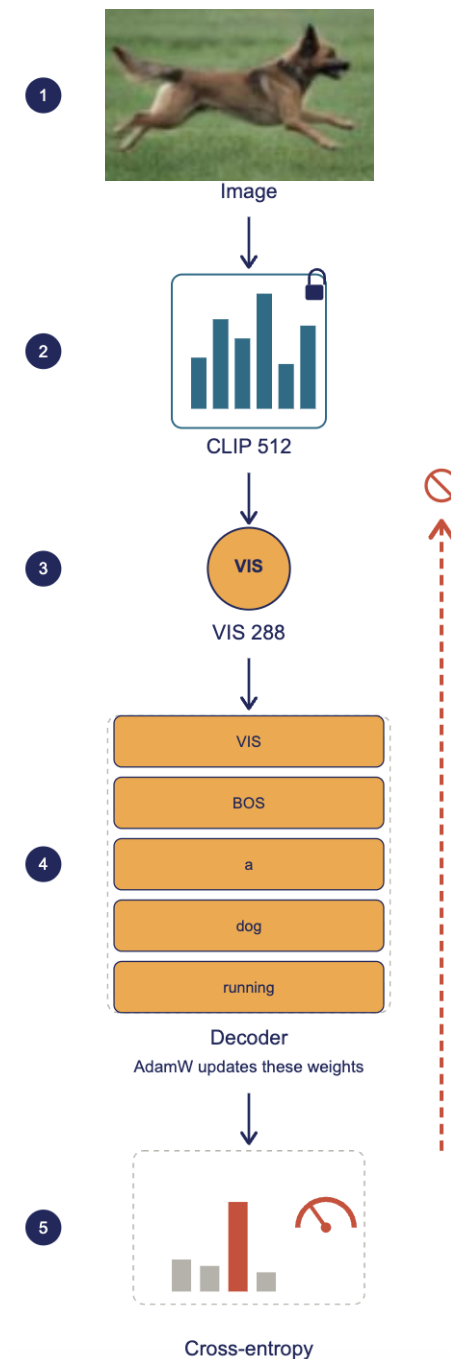
Cross-entropy measures the error

correctly aligned: `logits[:, 1:]` against `caption_ids[:, 1:]`.

5

Backprop updates decoder + projector

CLIP stays frozen throughout -- including through `model.train()`.



SCOPE, HONESTLY

This is a toy. Here's exactly why.

!

Scale

6-15M decoder params vs. billions in a real VLM

!

Data

1,500 training images (this demo) vs. hundreds of million

!

Tokenization

character-level, not a trained subword vocabulary

!

Visual detail

one pooled embedding, not per-patch cross-attention

!

No KV-cache

generation recomputes attention over the whole prefix each step

!

No formal metric

no BLEU/CIDEr -- judged by reading captions, not scoring them

THIS EXAMPLE



6M



1,500 images

t h e _ c a t _ s a t _ o n

Character-level tiles



1 pooled vector



Recompute full prefix



Eyeballed caption

PRACTICAL VLM



Billions



Millions+

the cat sat_on the mat

Subword tokens



Patch-grid tokens



Reused K/V cache



Metric + human review

From lecture demo toward something real

1

Bigger data needs a dataset-specific adapter

COCO/LAION aren't drop-in -- `tinyvlm_data.py` assumes Flickr8k's exact column names and official splits.

2

A real multimodal pretrained checkpoint

swap in an open small VLM's weights where freezing the decoder actually makes sense.

3

Exporting the decoder isn't exporting the model

`llama2.c's run.c` handles the decoder alone -- CLIP and the projector still need their own export path (e.g. ONNX/Core ML).

4

A real GPU box

same `trainVLM.py`, pointed at `cuda` instead of `cpu/mps` -- device is a speed knob here, not a correctness one.

Same recipe, more capable -- with the caveats included

1

Run the stories_{15M} comparison

seed the decoder from Karpathy's text-only checkpoint (tokenizer.model, not tokenizer.bin) and compare its loss curve to random init -- measure it, don't assume it.

2

Project more visual tokens

CLIP's patch-level features as a longer prefix -- still self-attention over a longer sequence, NOT automatically cross-attention.

3

Turn captioning into VQA

needs a real input/target convention and loss masking on the question tokens, not just a prepended question.

4

Freeze the decoder -- once it has a prior worth freezing

a standard alignment pattern: freeze a language-capable decoder (text-pretrained or multimodal), train only the projector. Freezing the default RANDOM decoder trains nothing useful.

THE STRONGEST DEMO YOU CAN RUN

Does it use the image, or recite a common sentence?

Condition	Visual token fed to the decoder
(a) Matched	this photo's real CLIP embedding
(b) Shuffled	a different photo's CLIP embedding
(c) Image-independent	no visual token at all



Same decoder, same checkpoint, same decoding settings -- only the visual token changes. If (a)'s caption differs meaningfully from (b) and (c), that's evidence the model is conditioning on the actual image, not just generating a generic Flickr-style sentence.

TAKE-HOME EXERCISES

Layered by year, so everyone has something to do

2

Annotate every tensor shape in `tinyvlm_model.py`

and calculate iterations-per-epoch for a batch size of your choice.

3

Find and re-fix the two bugs this talk covered

from a deliberately-reintroduced-bug copy of the code -- alignment and split leakage.

4

Run one ablation from the earlier slide

random vs. text-pretrained init, pooled vs. patch features, or frozen vs. trainable decoder -- bring a loss curve.

BACK TO THAT TERMINAL, ONE LAST TIME

Is it done? Take it home and break it.

```
[train] done. final checkpoint: checkpoints/last.pt  
best val_loss checkpoint: checkpoints/best.pt
```

```
$ python3 trainVLM.py --config config.demo.yaml --smoke-test
```

```
$ python3 inferVLM.py --config config.demo.yaml --checkpoint checkpoints/best.pt --holdout-index 0
```